Identificação do projeto

Título do projeto: PJBL Steam Analytics — preço, gênero, reviews e pico de jogadores simultâneos na Steam
Integrantes do grupo: Lucas Stopinski da Silva (RA 40041246) • Lucas Bruno e Silva (RA a confirmar)
Turma: Data Analytics (PjBL) • PUCPR • Turma U • 2026/2
Descrição breve do problema ou contexto: A Steam publica, para cada jogo, preço, gêneros, tags, reviews e uma estimativa de pico de jogadores simultâneos (Peak CCU). O projeto investiga quais características de um jogo acompanham maior audiência e como o catálogo histórico se compara ao comportamento dos jogos mais jogados em tempo real.

Notebook único do projeto, preenchido ao longo de RA1 e RA2. O detalhamento completo do RA1 está em `Notebooks/01_inspect_prepare.ipynb` e em `docs/RA1_Data_Analytics.docx`; aqui o fluxo foi condensado e é reexecutável do início ao fim.

RA1 — Data Ingestion, Cleaning and Initial Analysis

Nesta etapa, carregue, prepare e explore os dados do projeto.

1. Contexto e objetivo

Apresente brevemente o problema investigado e o que o grupo pretende descobrir com os dados.

Resposta do grupo:

A Steam é a maior loja de jogos para PC e tem um catálogo grande (mais de 125 mil títulos), heterogêneo e com muitos jogos sem telemetria de audiência. Desenvolvedoras e analistas querem saber **quais características de um jogo (preço, gênero/tags, nota de review) estão associadas a um pico maior de jogadores simultâneos (`Peak CCU`)** e como o catálogo histórico se compara ao comportamento ao vivo dos jogos mais jogados.

Perguntas analíticas do projeto: P1 (preço × `Peak CCU`), P2 (gêneros e tags com maior `Peak CCU`), P3 (nota de review e Metacritic × `Peak CCU`) e P4 (histórico × dado ao vivo). No RA1 o objetivo é apenas obter, entender e preparar o catálogo; as respostas ficam para as próximas etapas.

2. Fonte e ingestão dos dados

- Identifique a fonte dos dados, incluindo link ou referência de acesso.
- Descreva brevemente o dataset e o que seus registros representam.
- Explique como os dados foram carregados e implemente a ingestão.

Resposta do grupo:

Fonte principal: fronkongames/steam-games-dataset (Kaggle, acesso em 07/10/2026), arquivo `games.csv`, 401 MB, CSV UTF-8, **125.855 jogos × 40 campos**. Cada registro representa **um jogo ou aplicativo da Steam**, identificado por `AppID`.

Ingestão: o arquivo não vai para o Git (limite de 100 MB). Para obtê-lo: `pip install -r requirements.txt`, token em `~/.kaggle/kaggle.json` e `kaggle datasets download -d fronkongames/steam-games-dataset -p Dataset/fronkongames --unzip`. A célula abaixo valida a presença do arquivo e faz a leitura.

In [1]:
```
import csv
from pathlib import Path

import pandas as pd

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 160)

CAMINHO_ORIGINAL = Path("../Dataset/fronkongames/games.csv")
assert CAMINHO_ORIGINAL.exists(), (
    f"{CAMINHO_ORIGINAL} não encontrado. Baixe com o comando kaggle indicado acima."
)
print(f"Base original encontrada: {CAMINHO_ORIGINAL} ({CAMINHO_ORIGINAL.stat().st_size / 1e6:.0f} MB)")

# Leitura ingênua: serve de diagnóstico, o resultado sai desalinhado (ver seção 3).
df_bruto = pd.read_csv(CAMINHO_ORIGINAL)
print(f"Leitura ingênua: {df_bruto.shape[0]} linhas x {df_bruto.shape[1]} colunas")
```

```
Base original encontrada: ../Dataset/fronkongames/games.csv (401 MB)
Leitura ingênua: 125855 linhas x 39 colunas
```

3. Exploração inicial dos dados

Investigue a quantidade de registros e colunas, os tipos de dados, os valores ausentes e outras características relevantes da base.

In [2]:
```
# Cabeçalho x primeira linha de dados, via csv.reader (aspas tratadas
corretamente)
with open(CAMINHO_ORIGINAL, newline="", encoding="utf-8") as f:
    leitor = csv.reader(f)
    cabecalho = next(leitor)
    primeira_linha = next(leitor)

print(f"Campos no cabeçalho: {len(cabecalho)} | campos na 1ª linha de dado:
{len(primeira_linha)}")
print("Trecho do cabeçalho ao redor do ponto suspeito:", cabecalho[6:10])
print("Amostra de 'Release date' na leitura ingênua (deveria ser data):")
print(df_bruto["Release date"].sample(5, random_state=2).tolist())
print(f"AppID nulo na leitura ingênua: {df_bruto['AppID'].isnull().sum()}")
```

```
Campos no cabeçalho: 39 | campos na 1ª linha de dado: 40
Trecho do cabeçalho ao redor do ponto suspeito: ['Price', 'DiscountDLC count',
'About the game', 'Supported languages']
Amostra de 'Release date' na leitura ingênua (deveria ser data):
['200000 - 500000', '0 - 0', '0 - 20000', '0 - 20000', '0 - 20000']
AppID nulo na leitura ingênua: 1
```

In [3]:
```
# Releitura com cabeçalho corrigido: "DiscountDLC count" era a fusão de
"Discount" + "DLC count"
cabecalho_corrigido = cabecalho[:8] + ["Discount"] + cabecalho[8:]
df = pd.read_csv(CAMINHO_ORIGINAL, skiprows=1, header=None,
names=cabecalho_corrigido)
print(f"Leitura corrigida: {df.shape[0]} linhas x {df.shape[1]} colunas |
AppID nulo: {df['AppID'].isnull().sum()}")

total = len(df)
resumo = pd.DataFrame({
    "verificação": [
        "Coluna 100% vazia (Movies)",
        "Metacritic score == 0 (sentinela de 'sem nota')",
        "Genres ausentes", "Tags ausentes",
        "Linhas duplicadas (exatas)", "AppID duplicado", "Name repetido
(AppID distinto)",
        "Peak CCU == 0 (sem telemetria)",
    ],
    "resultado": [
        f"{df['Movies'].isna().sum()} de {total} nulos",
        f"{(df['Metacritic score'] == 0).sum()} ({(df['Metacritic score'] ==
0).mean():.1%})",
        f"{df['Genres'].isna().sum()} ({df['Genres'].isna().mean():.1%})",
        f"{df['Tags'].isna().sum()} ({df['Tags'].isna().mean():.1%})",
        int(df.duplicated().sum()), int(df["AppID"].duplicated().sum()),

int(df["Name"].astype(str).str.strip().str.lower().duplicated().sum()),
        f"{(df['Peak CCU'] == 0).sum()} ({(df['Peak CCU'] ==
0).mean():.1%})",
    ],
})
display(resumo.style.hide(axis="index"))
print("Estimated owners (texto em faixa):", df["Estimated
owners"].head(3).tolist())
```

```
Leitura corrigida: 125855 linhas x 40 colunas | AppID nulo: 0
```

verificação	resultado
Coluna 100% vazia (Movies)	125855 de 125855 nulos
Metacritic score == 0 (sentinela de 'sem nota')	121597 (96.6%)
Genres ausentes	8423 (6.7%)
Tags ausentes	42502 (33.8%)
Linhas duplicadas (exatas)	0
AppID duplicado	0
Name repetido (AppID distinto)	1593
Peak CCU == 0 (sem telemetria)	106175 (84.4%)

```
Estimated owners (texto em faixa): ['0 – 0', '0 – 20000', '0 – 20000']
```

Resposta do grupo:

- **Cabeçalho malformado (problema mais grave):** 39 nomes contra 40 campos por linha; `"DiscountDLC count"` é a fusão de `Discount` e `DLC count`. Na leitura ingênua quase todas as colunas ficam desalinhadas (`Release date` vem com faixas de owners, `AppID` tem 1 nulo).
- `Movies` está 100% vazia; `Estimated owners` é texto em faixa (`"0 – 20000"`); `Metacritic score` usa 0 como "sem nota" em 96,6% das linhas.
- Ausências em `Genres` , `Developers` , `Categories` , `Publishers` e `Tags` (de 6,7% a 33,8%).
- Sem duplicatas de linha ou de `AppID` ; 1.593 nomes repetidos com `AppID` distinto (demos, remasters).
- `Peak CCU` é 0 em 84,4% dos jogos: é **ausência de telemetria**, não pico zero. Valores extremos de `Price` e `Peak CCU` são reais (ex.: Counter-Strike 2 com 1.013.936).

Verificações completas em `Notebooks/01_inspect_prepare.ipynb` .

4. Limpeza e preparação dos dados

Realize a limpeza necessária ao projeto. Verifique valores ausentes, duplicados e inconsistências e trate os problemas encontrados, quando necessário.

In [4]:
```python
df_limpo = df.copy(deep=True)

# 1) Coluna Movies 100% vazia: sem informação, removida
df_limpo = df_limpo.drop(columns=["Movies"])

# 2) Metacritic score: 0 significa "sem nota", não nota mínima
df_limpo["Metacritic score"] = df_limpo["Metacritic score"].replace(0,
```

```
pd.NA)

# 3) Ausências em categóricas: marcador explícito mantém o jogo nos
agrupamentos
categoricas = ["Genres", "Developers", "Categories", "Publishers", "Tags"]
df_limpo[categoricas] = df_limpo[categoricas].fillna("Not specified")

# 4) Duplicatas exatas: verificação (a base não tem; AppID é a chave, Name
pode repetir)
df_limpo = df_limpo.drop_duplicates().reset_index(drop=True)

# 5) Outliers de Price/Peak CCU/Required age: mantidos, são valores reais;
só se checa o impossível
assert (df_limpo[["Price", "Peak CCU"]] >= 0).all().all() and
df_limpo["Required age"].between(0, 21).all()

print(f"Após a limpeza: {df_limpo.shape[0]} linhas x {df_limpo.shape[1]}
colunas")
print(f"Metacritic == 0: {(df_limpo['Metacritic score'] == 0).sum()} | NaN:
{df_limpo['Metacritic score'].isna().sum()}")
print(f"Ausentes em categóricas:
{int(df_limpo[categoricas].isna().sum().sum())}")
```

```
Após a limpeza: 125855 linhas x 39 colunas
Metacritic == 0: 0 | NaN: 121597
Ausentes em categóricas: 0
```

Resposta do grupo:

Problema	Tratamento	Efeito
Cabeçalho com 39 nomes para 40 campos	Cabeçalho reconstruído (inserido `Discount`)	125.855 linhas realinhadas, `AppID` sem nulos
`Movies` 100% vazia	Coluna removida	-1 coluna
`Metacritic score` == 0 em 121.597 linhas	0 convertido em NaN	médias passam a refletir só as 4.258 notas reais
Ausências em 5 colunas categóricas	Preenchidas com `"Not specified"`	nenhuma linha perdida em agrupamentos
Duplicatas exatas	Verificadas: 0	nada removido
Nomes repetidos (1.593)	Mantidos	`AppID` é a chave real
Outliers de `Price`, `Peak CCU`, `Required age`	Mantidos	são valores reais; `Peak CCU` == 0 não foi imputado

5. Transformação dos dados

Realize as transformações necessárias à análise, como criação de colunas, filtros, agrupamentos, conversões ou junções, quando aplicáveis.

In [5]:
```python
# Estimated owners: faixa em texto -> colunas numéricas
faixa = df_limpo["Estimated owners"].str.split(" - ", expand=True)
df_limpo["owners_min"] = pd.to_numeric(faixa[0], errors="coerce")
df_limpo["owners_max"] = pd.to_numeric(faixa[1], errors="coerce")
df_limpo["owners_avg"] = (df_limpo["owners_min"] + df_limpo["owners_max"]) /
2

# Release date: texto -> datetime
df_limpo["Release date"] = pd.to_datetime(df_limpo["Release date"],
errors="coerce", format="mixed")

print(f"owners_* ausentes:
{int(df_limpo[['owners_min','owners_max','owners_avg']].isna().sum().sum())}
")
print(f"Release date: {df_limpo['Release date'].dtype} | datas não
convertidas: {df_limpo['Release date'].isna().sum()}")

# Validação do conjunto preparado
assert df_limpo.shape == (125855, 42), df_limpo.shape
assert df_limpo["AppID"].is_unique and df_limpo["AppID"].notna().all()
assert "Movies" not in df_limpo.columns
print(f"Conjunto preparado: {df_limpo.shape[0]} linhas x {df_limpo.shape[1]}
colunas | AppID único")
```

```
owners_* ausentes: 0
Release date: datetime64[us] | datas não convertidas: 0
Conjunto preparado: 125855 linhas x 42 colunas | AppID único
```

Resposta do grupo:

`Estimated owners` virou três colunas numéricas (`owners_min`, `owners_max`, `owners_avg`, esta última o ponto médio da faixa) e `Release date` virou `datetime`, o que permite análises numéricas e temporais nas próximas etapas. A coluna original de owners foi mantida. Nenhuma agregação analítica foi feita neste RA.

7. Considerações do RA1

Registre brevemente:

- O que o grupo descobriu sobre os dados.
- Quais dificuldades ou limitações encontrou.
- Como os dados preparados poderão ser utilizados nas próximas etapas.

Resposta do grupo:

- **O que foi descoberto:** o dataset tinha um defeito estrutural (cabeçalho) que invalidaria qualquer análise; corrigido, a base fica íntegra (125.855 × 42, `AppID` único). 84,4% dos jogos não têm telemetria de `Peak CCU`.
- **Limitações:** as perguntas P1 a P3 valem só para o subconjunto com `Peak CCU` > 0 (cerca de 16%); Metacritic existe em 3,4% dos jogos; `owners_avg` é ponto médio

de faixa.
- **Próximas etapas:** `df_limpo` é o ponto de partida do RA2. Como `Peak CCU` é a variável mais confiável da base, ela serve de referência para simular, no RA2, a chegada contínua de leituras de jogadores dos jogos mais jogados.

RA2 — Streaming Data Processing and Time Windows

Utilize os dados e o contexto do projeto para demonstrar o processamento em streaming e a aplicação de janelas temporais. Escolha um dos cenários:

- **Cenário A — Dados estáticos:** simule a chegada de novos registros utilizando dados artificiais coerentes com o contexto do projeto.
- **Cenário B — Dados em streaming:** utilize sua própria fonte de dados contínuos, sem necessidade de simulação.

8. Objetivo da análise em streaming

Responda brevemente:

- O que o grupo deseja monitorar ou analisar?
- Qual informação espera obter com o processamento em streaming?

Resposta do grupo:

Queremos **monitorar, em janelas de 5 minutos, a média de jogadores simultâneos dos jogos mais jogados da Steam** enquanto novas leituras chegam. A informação esperada é como o nível de audiência de cada jogo evolui ao longo do tempo (tendência de subida ou queda), sem esperar o fim da coleta, que é a ideia do processamento em streaming. Isso responde, de forma simplificada, à pergunta P4 do projeto (histórico x comportamento ao vivo).

9. Fonte de dados e processamento

- Informe o cenário escolhido: dados reais em streaming ou simulação.
- Explique brevemente como os novos registros chegam.
- Implemente o processamento em tempo real.
- Demonstre o recebimento de novos registros.

Resposta do grupo:

Cenário A — dados estáticos com simulação. A base do projeto é um catálogo fixo; por isso as leituras que chegam em streaming são **artificiais, mas coerentes com a base**: pegamos os 3 jogos de maior `Peak CCU` de `df_limpo` e, para cada um, geramos uma leitura de jogadores simultâneos por minuto durante 15 minutos (45 eventos). O valor de cada leitura parte de uma fração do `Peak CCU` real do jogo, com tendência de alta e ruído pequeno, e nunca ultrapassa esse pico.

As leituras chegam em **3 lotes**, como arquivos JSON numa pasta monitorada pelo Spark Structured Streaming (`readStream` com schema explícito). Cada lote é escrito numa pasta de publicação e só então movido para a pasta de entrada, para o Spark nunca ler arquivo incompleto.

In [6]:
```
import json
import random
import tempfile
from pathlib import Path
from uuid import uuid4

from pyspark.sql import SparkSession
from pyspark.sql.functions import avg, round as arredonda, window
from pyspark.sql.types import IntegerType, StringType, StructField,
StructType, TimestampType

spark = (
    SparkSession.builder
    .master("local[2]")
    .appName("Steam streaming — RA2")
    .config("spark.sql.session.timeZone", "America/Sao_Paulo")
    .config("spark.sql.shuffle.partitions", "2")
    .getOrCreate()
)
spark.sparkContext.setLogLevel("ERROR")
print("Spark", spark.version, "| fuso da sessão:",
spark.conf.get("spark.sql.session.timeZone"))

# Os 3 jogos de maior Peak CCU da base preparada no RA1
top3 = df_limpo.nlargest(3, "Peak CCU")[["AppID", "Name", "Peak
CCU"]].reset_index(drop=True)
display(top3)
```

```
Spark 4.0.3 | fuso da sessão: America/Sao_Paulo
```

	AppID	Name	Peak CCU
0	730	Counter-Strike 2	1013936
1	570	Dota 2	623941
2	578080	PUBG: BATTLEGROUNDS	314682

In [7]:
```
# Geração das 45 leituras (3 jogos x 15 minutos). Determinística: mesma
semente, mesmos dados.
rng = random.Random(42)
```

```
INICIO = pd.Timestamp("2026-10-21 14:00:00")
MINUTOS = 15

leituras = []
for minuto in range(MINUTOS):
    for jogo in top3.to_dict("records"):
        fracao = min(0.70 + 0.02 * minuto + rng.uniform(-0.03, 0.03), 1.0)
# tendência de alta + ruído
        leituras.append({
            "timestamp": (INICIO +
pd.Timedelta(minutes=minuto)).strftime("%Y-%m-%d %H:%M:%S"),
            "appid": int(jogo["AppID"]),
            "jogo": jogo["Name"],
            "jogadores": int(jogo["Peak CCU"] * fracao),
        })

eventos = pd.DataFrame(leituras)
picos = top3.set_index("AppID")["Peak CCU"]
assert (eventos.groupby("appid")["jogadores"].max() <=
picos.reindex(sorted(picos.index))).all()
print(f"{len(eventos)} leituras geradas. Primeiras linhas:")
display(eventos.head(6))
```

```
45 leituras geradas. Primeiras linhas:
```

	timestamp	appid	jogo	jogadores
0	2026-10-21 14:00:00	730	Counter-Strike 2	718237
1	2026-10-21 14:00:00	570	Dota 2	418976
2	2026-10-21 14:00:00	578080	PUBG: BATTLEGROUNDS	216029
3	2026-10-21 14:01:00	730	Counter-Strike 2	713195
4	2026-10-21 14:01:00	570	Dota 2	458090
5	2026-10-21 14:01:00	578080	PUBG: BATTLEGROUNDS	229907

In [8]:
```
# Estrutura de pastas e fonte de streaming (padrão visto em aula: schema
explícito + pasta monitorada)
area_temporaria = tempfile.TemporaryDirectory(prefix="steam_stream_")
base = Path(area_temporaria.name)
pasta_entrada, pasta_publicacao, pasta_checkpoint = base / "entrada", base /
"publicacao", base / "checkpoint"
pasta_entrada.mkdir(); pasta_publicacao.mkdir()

schema = StructType([
    StructField("timestamp", TimestampType(), True),
    StructField("appid", IntegerType(), True),
    StructField("jogo", StringType(), True),
    StructField("jogadores", IntegerType(), True),
])

leituras_stream = (
    spark.readStream
    .schema(schema)
```

```
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss")
    .json(str(pasta_entrada))
)
print("É um DataFrame de streaming?", leituras_stream.isStreaming)
print("Arquivos na pasta de entrada:", list(pasta_entrada.iterdir()))
```

```
É um DataFrame de streaming? True
Arquivos na pasta de entrada: []
```

10. Aplicação de janelas temporais

- Utilize uma janela **Tumbling** (intervalos sem sobreposição) ou **Sliding** (intervalos que podem se sobrepor).
- Informe o tipo e a duração da janela e o deslocamento, quando aplicável.
- Implemente uma agregação coerente com a análise, como contagem, soma ou média.
- Explique brevemente o que será calculado em cada intervalo.

O foco é demonstrar a compreensão do conceito; não é necessário justificar tecnicamente a duração escolhida.

Resposta do grupo:

- **Tipo:** janela **Tumbling** (intervalos consecutivos, sem sobreposição).
- **Duração:** 5 minutos. **Deslocamento:** não se aplica (na tumbling o deslocamento é igual à duração).
- **Agregação: média** de `jogadores` (`avg`), calculada por janela **e por jogo**.
- **O que cada janela calcula:** a média de jogadores simultâneos de cada jogo nas leituras cujo `timestamp` cai naquele intervalo de 5 minutos. Início incluído, fim excluído: a leitura das 14:05:00 entra na janela 14:05-14:10. Cada leitura pertence a uma única janela.

In [9]:
```
medias_por_janela = (
    leituras_stream
    .groupBy(window("timestamp", "5 minutes"), "jogo")
    .agg(arredonda(avg("jogadores"),
0).cast("long").alias("media_jogadores"))
)

nome_tabela = "medias_" + uuid4().hex
consulta = (
    medias_por_janela.writeStream
    .format("memory")
    .queryName(nome_tabela)
    .outputMode("complete")          # a tabela de resultado é reescrita
inteira a cada lote
    .option("checkpointLocation", str(pasta_checkpoint))
    .start()
)
print("Consulta ativa:", consulta.isActive)
```

```
def resultado():
    return (
        spark.table(nome_tabela)
        .select(
            "window.start", "window.end", "jogo", "media_jogadores",
        )
        .orderBy("start", "jogo")
        .toPandas()
    )

def chegada_de_lote(numero, inicio_min, fim_min):
    lote = eventos.iloc[inicio_min * 3:(fim_min + 1) * 3]            # 3
jogos por minuto
    arquivo = pasta_publicacao / f"lote_{numero}_{uuid4().hex}.json"
    with arquivo.open("w", encoding="utf-8") as f:
        for linha in lote.to_dict("records"):
            f.write(json.dumps(linha) + "\n")
    arquivo.replace(pasta_entrada / arquivo.name)                     #
publica só com o arquivo já fechado
    consulta.processAllAvailable()
    print(f"Lote {numero}: {len(lote)} leituras (minutos {inicio_min} a
{fim_min}) | "
          f"última leitura recebida: {lote['timestamp'].iloc[-1]}")
    return lote
```

```
Consulta ativa: True
```

Cada lote abaixo simula uma chegada de dados em um momento diferente. Após cada um, mostramos a tabela de resultados: as janelas já completas permanecem iguais e a janela em curso é atualizada.

11. Resultados e interpretação

Apresente os resultados do processamento e demonstre como a análise se comporta com a chegada de novos registros.

In [10]:
```
# Lote 1 — minutos 0 a 6: a janela 14:00–14:05 fica completa e a 14:05–14:10
recebe só 2 minutos
chegada_de_lote(1, 0, 6)
res1 = resultado()
display(res1)
```

```
Lote 1: 21 leituras (minutos 0 a 6) | última leitura recebida: 2026-10-21
14:06:00
```

	start	end	jogo	media_jogadores
0	2026-10-21 14:00:00	2026-10-21 14:05:00	Counter-Strike 2	741931
1	2026-10-21 14:00:00	2026-10-21 14:05:00	Dota 2	452476
2	2026-10-21 14:00:00	2026-10-21 14:05:00	PUBG: BATTLEGROUNDS	232973
3	2026-10-21 14:05:00	2026-10-21 14:10:00	Counter-Strike 2	832067
4	2026-10-21 14:05:00	2026-10-21 14:10:00	Dota 2	490922
5	2026-10-21 14:05:00	2026-10-21 14:10:00	PUBG: BATTLEGROUNDS	258622

In [11]:
```
# Lote 2 — minutos 7 a 11: completa a janela 14:05–14:10 e abre a 14:10–14:15
chegada_de_lote(2, 7, 11)
res2 = resultado()
display(res2)
```

```
Lote 2: 15 leituras (minutos 7 a 11) | última leitura recebida: 2026-10-21 14:11:00
```

	start	end	jogo	media_jogadores
0	2026-10-21 14:00:00	2026-10-21 14:05:00	Counter-Strike 2	741931
1	2026-10-21 14:00:00	2026-10-21 14:05:00	Dota 2	452476
2	2026-10-21 14:00:00	2026-10-21 14:05:00	PUBG: BATTLEGROUNDS	232973
3	2026-10-21 14:05:00	2026-10-21 14:10:00	Counter-Strike 2	859085
4	2026-10-21 14:05:00	2026-10-21 14:10:00	Dota 2	518504
5	2026-10-21 14:05:00	2026-10-21 14:10:00	PUBG: BATTLEGROUNDS	263377
6	2026-10-21 14:10:00	2026-10-21 14:15:00	Counter-Strike 2	946415
7	2026-10-21 14:10:00	2026-10-21 14:15:00	Dota 2	569813
8	2026-10-21 14:10:00	2026-10-21 14:15:00	PUBG: BATTLEGROUNDS	287194

In [12]:
```
# Lote 3 — minutos 12 a 14: completa a janela 14:10–14:15
chegada_de_lote(3, 12, 14)
res3 = resultado()
display(res3)
```

```
Lote 3: 9 leituras (minutos 12 a 14) | última leitura recebida: 2026-10-21 14:14:00
```

	start	end	jogo	media_jogadores
0	2026-10-21 14:00:00	2026-10-21 14:05:00	Counter-Strike 2	741931
1	2026-10-21 14:00:00	2026-10-21 14:05:00	Dota 2	452476
2	2026-10-21 14:00:00	2026-10-21 14:05:00	PUBG: BATTLEGROUNDS	232973
3	2026-10-21 14:05:00	2026-10-21 14:10:00	Counter-Strike 2	859085
4	2026-10-21 14:05:00	2026-10-21 14:10:00	Dota 2	518504
5	2026-10-21 14:05:00	2026-10-21 14:10:00	PUBG: BATTLEGROUNDS	263377
6	2026-10-21 14:10:00	2026-10-21 14:15:00	Counter-Strike 2	964231
7	2026-10-21 14:10:00	2026-10-21 14:15:00	Dota 2	588157
8	2026-10-21 14:10:00	2026-10-21 14:15:00	PUBG: BATTLEGROUNDS	294198

In [13]:
```
# Comparação lado a lado: como cada janela mudou conforme os lotes chegaram
comparacao = (
    res1.rename(columns={"media_jogadores": "após lote 1"})
    .merge(res2.rename(columns={"media_jogadores": "após lote 2"}), on=
["start", "end", "jogo"], how="outer")
    .merge(res3.rename(columns={"media_jogadores": "após lote 3"}), on=
["start", "end", "jogo"], how="outer")
    .sort_values(["start", "jogo"])
)
comparacao[["após lote 1", "após lote 2", "após lote 3"]] =
comparacao[["após lote 1", "após lote 2", "após lote 3"]].astype("Int64")
comparacao["start"] = comparacao["start"].dt.strftime("%H:%M")
comparacao["end"] = comparacao["end"].dt.strftime("%H:%M")
display(comparacao.set_index(["start", "end", "jogo"]))

# Verificação: o resultado do streaming deve ser igual ao cálculo em batch
sobre os mesmos 45 eventos
batch = (
    eventos.assign(timestamp=pd.to_datetime(eventos["timestamp"]))
    .groupby([pd.Grouper(key="timestamp", freq="5min"), "jogo"])
["jogadores"].mean().round()
    .rename("media_jogadores").reset_index().rename(columns={"timestamp":
"start"})
    .sort_values(["start", "jogo"]).reset_index(drop=True)
)
final = res3[["start", "jogo", "media_jogadores"]].sort_values(["start",
"jogo"]).reset_index(drop=True)
assert (batch["media_jogadores"].astype("int64") ==
final["media_jogadores"].astype("int64")).all()
assert (batch["jogo"] == final["jogo"]).all()
print("Streaming e batch coincidem nas 9 médias (3 janelas x 3 jogos).")
```

			após lote 1	após lote 2	após lote 3
start	**end**	**jogo**			
14:00	**14:05**	**Counter-Strike 2**	741931	741931	741931
		Dota 2	452476	452476	452476
		PUBG: BATTLEGROUNDS	232973	232973	232973
14:05	**14:10**	**Counter-Strike 2**	832067	859085	859085
		Dota 2	490922	518504	518504
		PUBG: BATTLEGROUNDS	258622	263377	263377
14:10	**14:15**	**Counter-Strike 2**	<NA>	946415	964231
		Dota 2	<NA>	569813	588157
		PUBG: BATTLEGROUNDS	<NA>	287194	294198

```
Streaming e batch coincidem nas 9 médias (3 janelas x 3 jogos).
```

Resposta do grupo:

- **Janela 14:00-14:05 (completa no lote 1):** não muda mais depois disso (Counter-Strike 2: 741.931; Dota 2: 452.476; PUBG: 232.973). Uma janela fechada só mudaria se chegassem leituras atrasadas para ela.
- **Janela 14:05-14:10 (em curso):** após o lote 1 só tinha os minutos 5 e 6, e a média do Counter-Strike 2 era 832.067; com os minutos 7 a 9 do lote 2 passou a 859.085. Dota 2 foi de 490.922 para 518.504 e PUBG de 258.622 para 263.377. É a mudança que mostra o streaming funcionando: **o resultado é recalculado quando novos registros chegam**, sem reprocessar tudo.
- **Janela 14:10-14:15:** só existe a partir do lote 2 (minutos 10 e 11) e é ajustada no lote 3 (Counter-Strike 2: 946.415 para 964.231; Dota 2: 569.813 para 588.157; PUBG: 287.194 para 294.198).
- **Tendência:** as médias sobem de janela em janela nos três jogos, coerente com a tendência de alta usada na simulação (e sempre abaixo do `Peak CCU` real de cada jogo).
- **Conferência:** o resultado final do streaming é idêntico ao cálculo em batch sobre as mesmas 45 leituras (verificação com `assert`), o que valida o processamento.

In [14]:
```
# Encerramento: para a consulta e remove a tabela e as pastas temporárias
consulta.stop()
spark.catalog.dropTempView(nome_tabela)
area_temporaria.cleanup()
print("Consulta ativa:", consulta.isActive)
```

```
Consulta ativa: False
```

12. Considerações do RA2

Responda brevemente:

- O que foi possível observar com o processamento em streaming?
- Qual foi a contribuição das janelas temporais para a análise?
- Essa abordagem faria sentido em uma aplicação real do projeto? Por quê?

Na última resposta, o grupo pode reconhecer que o streaming foi utilizado apenas como exercício didático, se esse for o caso.

Resposta do grupo:

- **O que foi possível observar:** o Spark Structured Streaming trata a pasta de entrada como uma tabela sem limite e atualiza as médias a cada lote (micro-batch). Só a janela em curso muda; as já completas ficam estáveis.
- **Contribuição das janelas temporais:** agrupar por intervalos de 5 minutos transforma leituras soltas em um indicador comparável no tempo (média por janela), e deixa claro quando uma janela está completa ou ainda parcial. A tumbling foi suficiente porque cada leitura pertence a um único bloco; uma sliding suavizaria a curva, ao custo de repetir leituras entre janelas.
- **Faria sentido numa aplicação real?** Sim, em parte. Monitorar jogadores simultâneos de poucos jogos quase em tempo real (alertas de queda ou pico, acompanhamento de lançamento) combina com streaming, alimentado por um coletor da Steam Web API ou do SteamSpy no lugar da simulação. Para a análise histórica do catálogo (P1 a P3), o batch continua mais adequado. Neste trabalho o streaming foi um **exercício didático**: os dados são simulados a partir do `Peak CCU` real, não leituras reais.